Análise de Eficiência Energética e Desempenho da Paralelização do Processamento de Imagens com Filtros Convolucionais

Enzo de Almeida Belfort Rizzi Di Chiara[1], **Luiza de Souza Ferreira**[1], **Natália Vitória Amaral do Val**[1], **Vinícius Augusto Landim Andrade**[1], **Viviane Flor Park**[1]

[1]Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
São José dos Campos – SP – Brasil

{belfort.enzo, souza.luiza, amaral.natalia, vinicius.landim, viviane.park}@unifesp.br

Abstract. *This article describes the implementation of a parallelized version of a convolutional filter algorithm and the corresponding analysis of its energy efficiency and performance. The algorithm is parallelized in two different ways – using the OpenMP library and the CUDA platform – and each version is tested with a set of images of varying dimensions. The results of the parallel implementations are compared against the sequential version of the algorithm and against each other in terms of execution time, speedup, and energy efficiency.*

Resumo. *Este artigo descreve a implementação de uma versão paralelizada de um algoritmo de filtro convolucional e sua respectiva análise de eficiência energética e desempenho. A paralelização do algoritmo é feita de duas maneiras diferentes, com a biblioteca OpenMP e a plataforma CUDA, e cada versão é testada com um conjunto de imagens com diferentes dimensões. Os resultados das implementações paralelas são comparados com a versão sequencial do algoritmo e entre si em termos de tempo de execução, speedup e eficiência energética.*

1. Introdução

O crescimento exponencial no volume de dados visuais e a necessidade de processamento em tempo real tornaram o uso de filtros convolucionais uma tarefa de alto custo computacional. Embora a paralelização seja a principal estratégia para acelerar a execução, a eficiência energética tornou-se um fator crítico no projeto de sistemas modernos. O aumento do número de *threads* e o uso intensivo de *hardware* nem sempre geram ganho proporcional de velocidade (*speedup*), podendo elevar drasticamente o consumo de energia devido a gargalos de memória e contenção de barramento.

Este trabalho tem como objetivo, portanto, avaliar o *trade-off* entre ganho de desempenho e eficiência energética na paralelização de filtros convolucionais. Para isso, são analisados diferentes algoritmos e estratégias de concorrência aplicados a imagens de variados tamanhos, mensurando o tempo de execução e o consumo energético do processador. Com isso, busca-se identificar os limites de escalabilidade em que a paralelização otimiza o desempenho mantendo a sustentabilidade energética da aplicação.

2. Fundamentação Teórica e Trabalhos Relacionados

No contexto de processamento de imagens, um filtro convolucional é um mecanismo matemático que opera uma matriz de pesos, chamada de máscara ou *kernel*, sobre os píxeis de uma imagem, alterando os valores originais com base na combinação linear da vizinhança. Matematicamente, essa operação pode ser definida pela Equação 1, em que $f(x, y)$ representa a imagem de entrada, $g(x, y)$ representa a imagem de saída, (x, y) são as coordenadas do píxel da imagem de saída, (s, t) são os índices de posição na máscara convolucional, $w(s, t)$ representa o peso da máscara na posição (s, t) e m e n correspondem, respectivamente, à altura e à largura da máscara [Gonzalez and Woods 2010].

$$g(x, y) = \sum_{s=0}^{m-1} \sum_{t=0}^{n-1} w(s, t) f(x + s, y + t), \tag{1}$$

Devido ao seu elevado número de operações aritméticas repetitivas e à independência entre a computação de píxeis distintos, esta técnica apresenta uma elevada intensidade computacional e um alto grau de paralelismo de dados inerente, tornando-se ideal para arquiteturas concorrentes [Hwu et al. 2023].

Para explorar este paralelismo, utiliza-se, por exemplo, a biblioteca OpenMP e a plataforma CUDA. O OpenMP é uma biblioteca da linguagem C que possibilita a paralelização de tarefas em sistemas de memória compartilhada. Essa biblioteca encarrega o compilador de distribuir a carga computacional em *threads* que, por sua vez, podem ser processadas em diferentes *cores* da CPU [Dagum and Menon 1998]. O CUDA, por outro lado, é uma plataforma da NVIDIA que possibilita a utilização de GPUs para processamento de propósito geral. Dessa maneira, os milhares de núcleos de processamento desse componente passam a executar blocos de *threads* de forma altamente concorrente, reduzindo significativamente a latência de processamento em grandes volumes de dados [Hwu et al. 2023].

As soluções paralelizadas são frequentemente analisadas e validadas em termos de tempo de execução, *speedup* e, mais recentemente, da sua eficiência energética. O tempo de execução mede a duração total do processamento paralelo e pode ser obtido de forma fácil com funções de auxílio das próprias plataformas utilizadas ou do terminal do sistema operacional da máquina. O *speedup*, por sua vez, é uma métrica que quantifica o ganho em tempo do algoritmo paralelizado (com tempo de execução T_P em P *cores*) em relação à versão sequencial (com tempo de execução T em 1 *core*), e é calculado pela Equação 2 [Pacheco 2011].

$$S_P = \frac{T}{T_P} \tag{2}$$

Em termos de eficiência energética, uma métrica comum é o *energy-delay product* (EDP). Sua utilização é interessante pois relaciona a energia E gasta por um processo com o seu tempo T de execução, de acordo com a Equação 3 [Fratin et al. 2018]. Dessa maneira, é possível investigar o *trade-off* entre o aumento do grau de paralelização de um programa, o que pode diminuir seu tempo total de execução, com a energia utilizada. De maneira geral, um menor EDP é ideal.

$$EDP = E \cdot T \tag{3}$$

No domínio dos filtros convolucionais, tem-se que sua paralelização é uma abordagem consolidada na literatura, com implementações tanto em CPUs *multicore* quanto em GPUs. Quanto às métricas avaliadas, Afif et al. [Afif et al. 2018], por exemplo, implementaram em CUDA os filtros de Sobel e gaussiano e compararam seus tempos de execução com os de versões sequenciais em CPU, observando que o ganho cresce com o tamanho da imagem. Já Cremel e Arruda [Cremel and Arruda 2026] avaliaram a paralelização com OpenMP de cinco filtros de pré-processamento, medindo tempo de execução, *speedup* e eficiência paralela, e mostraram que o aumento do número de threads eleva o *speedup*, mas reduz a eficiência paralela.

Em ambos os trabalhos, porém, a avaliação se concentra em métricas de tempo e *speedup*, e o consumo de energia não é analisado. Quando o consumo energético é considerado, a convolução costuma aparecer no contexto das redes neurais convolucionais, em que a eficiência energética é avaliada em CPUs e GPUs [Xu et al. 2023, Li et al. 2016]. Nesses estudos, a análise tende a se dar no nível da rede ou de suas camadas, e não do filtro convolucional como operação isolada. Nesse contexto, o presente trabalho se diferencia por analisar o desempenho e a eficiência energética da convolução associados às técnicas de paralelização, comparando OpenMP e CUDA e três estratégias de uso da memória da GPU.

3. Metodologia e Implementação

O trabalho desenvolvido pode ser encontrado no repositório do github. Estruturalmente, o algoritmo sequencial aplicado é dividido em duas partes: o pré-processamento, que transforma a imagem em tons de cinza, reduzindo sua representação a um único canal, e a convolução, que adota um filtro de suavização pela média de dimensões 31x31. Em cima desse algoritmo, foram implementadas as versões paralelas com OpenMP e CUDA, detalhadas nas subseções 3.1 e 3.2, respectivamente.

Os testes foram executados em cima de um banco de 100 imagens, classificadas uniformemente em 5 dimensões diferentes: VGA (640×480), HD (1280×720), Full HD (1920×1080), QHD (2560×1440) e 4K (3840×2160). Essa execução foi feita em um computador com um processador Intel Core i5-9400F com frequência de *clock* de 2.90GHz e 6 *cores*. O computador possui 16GB de memória RAM DDR4 e uma placa de vídeo NVIDIA GeForce GTX 1060 de 6GB.

3.1. Paralelização com OpenMP

A paralelização do algoritmo com OpenMP foi realizada em dois momentos distintos: na parte de pré-processamento e de convolução. As duas se tornaram fortes candidatas à paralelização devido à independência entre o processamento de píxeis diferentes.

Tanto no pré-processamento, quanto na convolução da imagem, foi utilizada a diretiva `#pragma omp parallel for schedule(static)`. Ela permite distribuir as iterações do *loop* mais externo entre as *threads* de forma equitativa, de modo que diferentes píxeis possam ser processados em paralelo.

Para chegar na versão final da implementação com OpenMP, foram realizados testes de tempo de execução das funções, comparando diferentes configurações de paralelização. A configuração apresentada foi escolhida com base nos menores tempos de execução obtidos.

3.2. Paralelização com CUDA

As paralelizações usando CUDA basearam-se nas estratégias projetadas e descritas por Hwu et al. [Hwu et al. 2023]. Dentre elas, três abordagens foram implementadas, cada uma explorando diferentes níveis da hierarquia de memória da GPU. A primeira consiste em uma abordagem ingênua, caracterizada pelo acesso extensivo de leitura e escrita diretamente na memória global; a segunda abordagem aloca o filtro na *constant memory* e, por fim, a terceira estratégia emprega a técnica de *tiling*, permitindo o uso da memória compartilhada para reduzir acessos redundantes à memória global durante o cálculo dos píxeis de saída.

3.2.1. Paralelização base

Nesta abordagem inicial, a responsabilidade de calcular cada pixel da imagem de saída é atribuída individualmente a uma *thread* distinta. Para isso, as *threads* são organizadas em um *grid* bidimensional que recobre toda a dimensão da imagem, subdividido estruturalmente em blocos compostos por 16×16 *threads*. A identificação exata de qual elemento da matriz de saída será processado por cada *thread* é determinado a partir dos índices e dimensões do bloco e do índice da *thread*.

Apesar de funcional, essa estratégia apresenta restrições de desempenho. O primeiro obstáculo é a divergência no fluxo de controle, causada pelo tratamento das bordas da imagem, onde a janela do filtro ultrapassa os limites da matriz, exigindo verificações condicionais para evitar acessos inválidos à memória. Como resultado, *threads* de um mesmo *warp* podem seguir caminhos distintos, pois algumas executam os cálculos enquanto outras os ignoram. No modelo SIMT das GPUs, isso leva à serialização das instruções, reduzindo o paralelismo.

O segundo gargalo é a ineficiência no acesso à memória. Como as janelas de convolução vizinhas se sobrepõem quase inteiramente, um mesmo pixel de entrada é lido repetidamente por várias *threads*. O mesmo ocorre com os coeficientes do filtro, que são lidos por todas as *threads*. Além dessa redundância, todas os acessos são feitos na memória global, que possui alta latência e largura de banda limitada [Hwu et al. 2023]. Como consequência, o desempenho passa a ser limitado pela vazão da memória, e não pela capacidade de cálculo da GPU.

3.3. Paralelização usando a *constant memory* e *caching*

Para reduzir a quantidade de acessos à memória global, uma alternativa é declarar o filtro como um elemento a ser armazenado na *constant memory* da GPU. Segundo Hwu et al. [Hwu et al. 2023], o uso desse espaço de memória é viável e arquiteturalmente ideal devido a três características inerentes ao filtro convolucional. Primeiramente, os seus pesos numéricos permanecem inalterados durante toda a execução do *kernel*, o que

respeita a restrição dessa memória ser do tipo *read-only*. Além disso, a matriz do filtro possui dimensões reduzidas, de modo que os dados ocupam apenas alguns *kilobytes*, encaixando-se no limite de capacidade de 64 KB típico dessa memória.

A terceira razão diz respeito ao uso da cache associada à *constant memory*. Embora essa memória resida fisicamente na DRAM do dispositivo, seus dados são acessados por meio de caches especializadas em cada SM, e o ganho depende de os acessos encontrarem o dado já armazenado nelas. A convolução favorece isso em dois aspectos. Primeiro, os índices usados para ler o filtro dependem apenas dos laços de repetição, e não da *thread*, de modo que todas as *threads* de um *warp* acessam o mesmo elemento ao mesmo tempo, e uma única busca na DRAM atende a todas elas. Segundo, como o filtro é pequeno, ele cabe inteiro na cache, de modo que, após as primeiras leituras, os acessos seguintes são atendidos por ela, sem retornar à DRAM [Hwu et al. 2023].

3.3.1. Paralelização usando *tiling*

Uma característica inerente à operação de convolução é que, para calcular o valor de dois píxeis de saída adjacentes, o filtro se desloca apenas uma posição, de modo que é necessária a leitura de píxeis de entrada repetidos, com apenas uma fração de novos dados. Nas abordagens anteriores esse problema não foi considerado, de modo que a imagem de entrada era lida da memória global, e cada reutilização de um mesmo pixel resultava em uma nova requisição a essa memória [Hwu et al. 2023].

Sendo assim, a técnica de *tiling* surge como uma solução de desempenho. Essa abordagem consiste em particionar o grande volume de dados globais em subconjuntos menores, de modo a permitir sua alocação em uma memória menor e de acesso mais rápido, como a memória compartilhada. Aplicando o *tiling*, o segmento da imagem necessário às *threads* de um bloco é carregado uma única vez da memória global para a memória compartilhada, e as leituras repetidas passam a ser atendidas por essa memória interna, de acesso significativamente mais rápido.

Nesta terceira implementação, a técnica de *tiling* foi aplicada na memória compartilhada, mantendo o filtro na *constant memory*. Para viabilizar essa arquitetura, uma condição estrutural é que o *tile* de entrada seja estritamente maior que o *tile* de saída. Isso se deve à necessidade de incluir margens em todas as direções com largura igual ao raio do filtro, garantindo os dados necessários para o cálculo dos píxeis situados nas extremidades do bloco de saída.

A respeito da implementação, como escolha de projeto, os blocos de *threads* possuem as mesmas dimensões do *tile* de saída, atribuindo a cada *thread* o cálculo de um único píxel. Contudo, como o *tile* de entrada é maior que o de saída, cada *thread* pode precisar carregar múltiplos elementos da memória global para a compartilhada. Sendo assim, somente após o carregamento e sincronização do bloco a convolução é executada utilizando os dados locais. Embora essa etapa inicial tenha um custo adicional, a redução do tráfego na memória global promove a melhora do desempenho [Hwu et al. 2023].

3.4. Ambiente de Compilação e Coleta de Métricas

Para garantir o rigor na extração dos dados de tempo e energia, o ambiente de desenvolvimento C/C++ foi inicializado através do `vcvarsall.bat` (MSVC). Os ex-

ecutáveis baseados em CUDA foram gerados com o compilador `nvcc`, definindo-se a arquitetura de *hardware* específica da GPU (`-arch=sm_61`) e garantindo a compatibilidade da ferramenta (`-allow-unsupported-compiler`).

A energia da CPU foi obtida por RAPL e a da GPU pela biblioteca NVML, com período de amostragem de 2 ms. Além do *speedup* S_P (Equação 2), utilizam-se a eficiência paralela, $E_P = S_P/P$, e o produto energia-atraso (EDP $= E \cdot t$), que pondera o consumo de energia pelo tempo de execução. Além disso, para permitir a medição do consumo energético em tempo real (em Joules) diretamente dos registadores da placa gráfica, usou-se a flag (`-lnvml`) no compilador. Toda a bateria de testes operacionais foi automatizada por intermédio de um *script* em Python (`coleta_metricas.py`), responsável por submeter iterativamente o *dataset* de imagens aos executáveis e consolidar as métricas calculadas em tabelas e gráficos estruturados.

4. Resultados e Discussão

Nesta seção, avalia-se o desempenho e o consumo energético do filtro convolucional em duas frentes: (i) a aceleração obtida com a CPU *multicore* (OpenMP) e com a GPU (CUDA); e (ii) a sensibilidade da versão OpenMP ao volume de dados, variando-se a resolução da imagem de entrada. Os resultados referem-se a imagens 4K (3840×2160, 20 imagens) e a um filtro de $K \times K$ com $K = 31$, o que corresponde a uma contagem nominal de $2K^2 = 1922$ operações de ponto flutuante por pixel.

4.1. Desempenho e Eficiência Energética (Dataset 4K)

Na CPU, o OpenMP apresentou escalabilidade próxima da linear até 6 núcleos físicos. Em relação à versão sequencial ($11{,}41$ s), a execução com 6 *threads* ($2{,}04$ s) atingiu *speedup* de $5{,}36\times$ e eficiência paralela de $0{,}89$. Com uma única *thread*, a versão paralela foi 8% mais lenta que a sequencial ($0{,}92\times$), possivelmente devido ao custo da região paralela. A energia diminuiu de $464{,}04$ J (1 *thread*) para $152{,}98$ J (6 *threads*), redução de $3{,}0\times$, menor que a redução de $5{,}8\times$ no tempo, pois a potência média estimada (E/t) aumentou de $39{,}2$ W para $75{,}1$ W. Não se observou aumento do consumo de energia com o número de *threads*, o que é coerente com uma carga limitada por computação, e não por memória, nesta implementação.

A dispersão dos tempos (Figura 1) foi baixa entre 2 e 5 *threads* (coeficiente de variação inferior a $1{,}4\%$), mas maior com 1 e 6 *threads* ($5{,}7\%$ e $6{,}6\%$). Com 6 *threads*, uma possível causa é a interferência do sistema operacional, já que todos os núcleos físicos ficam ocupados.

Com a GPU, o ganho é de outra ordem de grandeza (Tabela 1). Em relação ao melhor caso da CPU (OpenMP, 6 *threads*), a versão CUDA *Tiled* foi $75{,}9\times$ mais rápida e consumiu $113{,}9\times$ menos energia; contra a versão sequencial, o *speedup* foi de $407{,}0\times$.

A evolução entre as versões CUDA pode ser resumida em três observações:

1. *Memória constante.* O tempo reduziu-se de $0{,}0817$ s para $0{,}0599$ s ($1{,}36\times$). O resultado é consistente com o mecanismo de *broadcast* da *constant cache*, que atende simultaneamente aos pesos do filtro lidos por todas as *threads* de um *warp*.
2. *Memória compartilhada (tiling).* A versão *Tiled* ($0{,}0268$ s) foi $2{,}23\times$ mais rápida que a de memória constante e $3{,}05\times$ mais rápida que a versão base. Esse ganho é consistente

Tabela 1. Comparativo de desempenho e eficiência energética (dataset 4K). O *speedup* é calculado em relação à versão sequencial e a Pot. é estimada por E/t.

Abordagem	Tempo (s)	Energia (J)	Pot. (W)	Speedup	EDP (J·s)
Sequencial	11,4100	587,40	51,48	1,00×	6696,36
OpenMP 1T	11,84	464,04	39,2	0,92×	5492,43
OpenMP 6T	2,04	152,98	75,1	5,36×	311,53
CUDA Global	0,0817	7,51	91,9	133,6×	0,613
CUDA Constante	0,0599	5,76	96,2	182,3×	0,345
CUDA *Tiled*	0,0268	1,34	50,1	407,0×	0,036

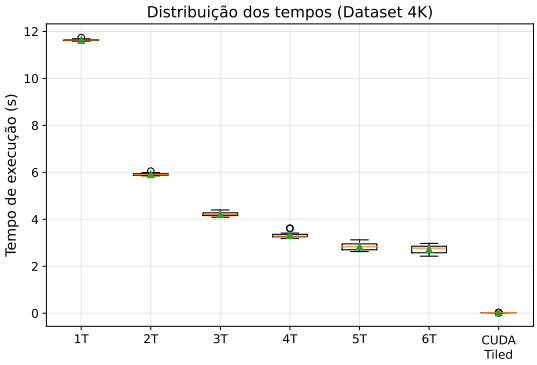


Figura 1. Distribuição dos tempos (4K).

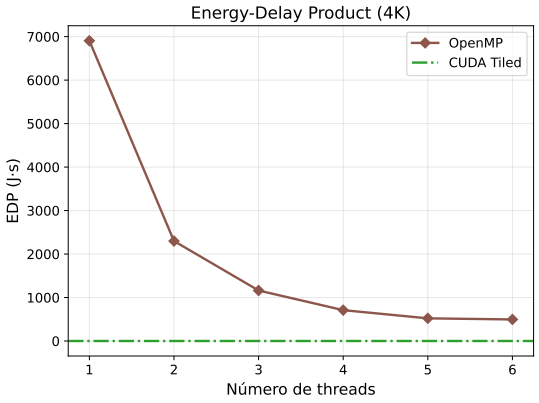


Figura 2. Energy-Delay Product (4K).

com a redução de leituras redundantes à memória global, já que cada pixel de entrada passa a ser carregado uma vez por bloco, e não uma vez por janela de convolução.

3. *Energia.* Como $E = \bar{P} \cdot t$, a redução de energia depende do tempo e da potência média. A potência média estimada foi de $91{,}9\,\mathrm{W}$ (Global), $96{,}2\,\mathrm{W}$ (Constante) e $50{,}1\,\mathrm{W}$ (*Tiled*), valores da mesma ordem de grandeza da estimada para a CPU com 6 *threads* ($75{,}1\,\mathrm{W}$). A razão de energia entre as versões Global e *Tiled* ($5{,}6\times$) supera a razão de tempo ($3{,}05\times$): o fator adicional de $1{,}8\times$ decorre da menor potência média da versão *Tiled*, possivelmente associada à menor demanda sobre a DRAM. Além disso, a energia foi integrada apenas durante a execução; o efeito de *race-to-idle* (maior tempo em repouso dentro de uma janela fixa) não está incluído nesses valores.

Essa superioridade consolida-se no EDP (Figura 2): com apenas $0{,}036\,\mathrm{J{\cdot}s}$, o *CUDA Tiled* atenuou o custo operacional em mais de $180.000\times$ em comparação à CPU, viabilizando uma eficiência superior a $12.000\,\mathrm{MFLOPS/Watt}$.

4.2. Impacto da Variação de Resolução (Volume de Dados)

Para investigar a sensibilidade dos algoritmos, a Figura 3 analisa o impacto desde a resolução VGA (640×480) até 4K.

Resoluções reduzidas (VGA) apresentam variações de escalabilidade provocadas pelo custo relativo de criação das *threads* e tráfego no barramento PCIe. Contudo, à medida que a dimensão escala, a curva de aceleração (Figura 3a) converge de forma linear para o comportamento ideal. No gráfico logarítmico (Figura 3b), constata-se que o

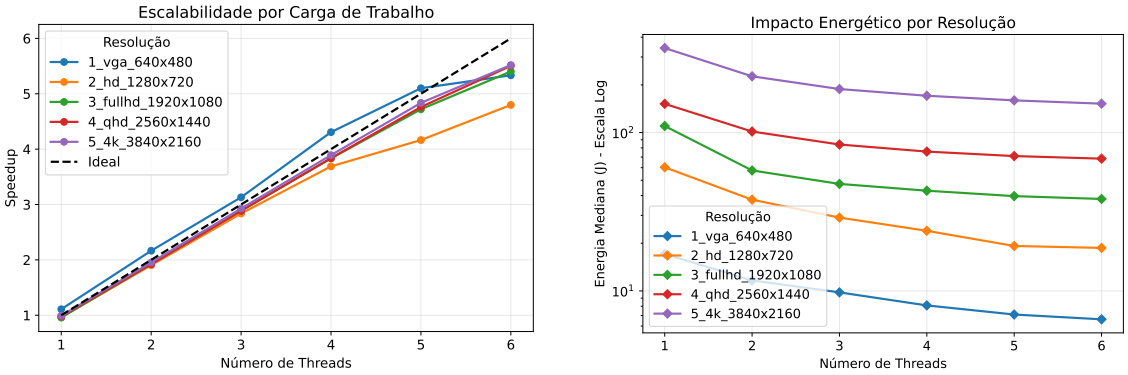


Figura 3. Impacto da resolução na CPU (OpenMP): (a) escalabilidade do *speedup*; (b) consumo energético mediano (escala logarítmica).

ganho energético se mantém válido em todas as ordens de grandeza. O paralelismo massivo dilui o custo das transferências de dados perante grandes volumes, comprovando que a aceleração concorrente é o mecanismo primário para assegurar a sustentabilidade energética em operações de elevada intensidade computacional.

5. Conclusão

Este trabalho analisou o *trade-off* entre desempenho e eficiência energética na paralelização de filtros convolucionais, comparando uma versão sequencial, uma implementação com OpenMP variando o número de *threads* entre 1 e 6 em CPU *multi-core* e três estratégias em CUDA que exploram diferentes níveis da hierarquia de memória da GPU. Todas as abordagens foram submetidas a um *dataset* composto por imagens de cinco resoluções distintas, do VGA (640×480) ao 4K (3840×2160), permitindo observar o comportamento dos algoritmos sob diferentes volumes de dados.

Os resultados mostraram que, em todas as configurações avaliadas, a redução do tempo de execução foi acompanhada por uma redução proporcional no consumo energético, de modo que a configuração mais rápida foi também a mais eficiente em cada arquitetura. Na CPU, o aumento do número de *threads* elevou o *speedup* de forma consistente, atingindo o melhor resultado com as 6 *threads* físicas disponíveis, com $5{,}36\times$ e $152{,}98\,\text{J}$ no *dataset* 4K. Na GPU, a mitigação progressiva dos gargalos de memória levou a ganhos crescentes, sendo $139{,}7\times$ na versão base, $190{,}6\times$ com o uso da *constant memory* e $425{,}5\times$ na implementação com *tiling* e memória compartilhada, que consumiu apenas $1{,}34\,\text{J}$. Essa queda simultânea de tempo e energia se traduz em um EDP mais de $180.000\times$ inferior ao da execução sequencial.

A variação da resolução evidenciou ainda que a eficiência da paralelização cresce com o volume de dados. Em resoluções baixas, o custo relativo de criação de *threads* e de tráfego de dados limita o ganho, mas, à medida que a dimensão da imagem aumenta, a curva de *speedup* converge para o comportamento ideal e o ganho energético se mantém válido em todas as ordens de grandeza. Como trabalhos futuros, pretende-se ampliar o número de *threads* utilizadas na versão OpenMP, uma vez que a análise atual foi limitada às 6 *threads* físicas disponíveis no *hardware* de teste, estender a avaliação a filtros convolucionais de maiores dimensões e investigar o comportamento das abordagens em outros cenários de execução, como GPUs de arquiteturas mais recentes.

Referências

[Afif et al. 2018] Afif, M., Said, Y., and Atri, M. (2018). Efficient 2d convolution filters implementations on graphics processing unit using nvidia cuda. *International Journal of Image, Graphics and Signal Processing*, v. 8.

[Cremel and Arruda 2026] Cremel, G. and Arruda, L. F. (2026). High-performance parallel acceleration of image processing filters on multicore cpus using openmp. *Anais do Computer on the Beach*, v. 17.

[Dagum and Menon 1998] Dagum, L. and Menon, R. (1998). Openmp: an industry standard api for shared-memory programming. *IEEE Computational Science and Engineering*, 5(1):46–55.

[Fratin et al. 2018] Fratin, V., Oliveira, D., Navaux, P., Carro, L., and Rech, P. (2018). Energy-delay-fit product to compare processors and algorithm implementations. *Microelectronics Reliability*, v. 84.

[Gonzalez and Woods 2010] Gonzalez, R. and Woods, R. (2010). *Processamento digital de imagens*. Pearson Education do Brasil, São Paulo, 3 edition.

[Hwu et al. 2023] Hwu, W. W., Kirk, D. B., and Hajj, E. I. (2023). *Programming Massively Parallel Processors*. Morgan Kaufmann, 4th edition.

[Li et al. 2016] Li, D., Chen, X., Becchi, M., and Zong, Z. (2016). Evaluating the energy efficiency of deep convolutional neural networks on cpus and gpus. In *2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom) (BDCloud-SocialCom-SustainCom)*, pages 477–484.

[Pacheco 2011] Pacheco, P. S. (2011). *An introduction to parallel programming*. Morgan Kaufmann, Burlington, EUA, 2nd edition.

[Xu et al. 2023] Xu, Y., Martínez-Fernández, S., Martinez, M., and Franch, X. (2023). Energy efficiency of training neural network architectures: An empirical study. *arXiv preprint arXiv:2302.00967*.